Use Cases

1. **Create a new project:** the developer creates a new project
2. **Load textures:** the developer loads base or custom textures for towers, enemies, strategic assets, GUI, background (map) and projectiles.
3. **Load font:** the developer loads base or custom font for GUI.
4. **Remove a texture:** the developer removes a texture
5. **Remove a font:** the developer removes a font
6. **Create an entity in project-global entities list:** the developer creates an entity (e.g. tower type, enemy type or some UI element) in project-global entities list.
7. **Create a level entity:** the developer creates an entity (e.g. tower spot, enemy wave) in currently opened level
8. **Remove an entity:** the developer removes an entity
9. **Configure wave entity:** the developer sets wave duration, next wave reference for wave, spawn rate, types of enemies for each enemy path involved in wave.
10. **Handle error:** the editor displays an error message.
11. **Get confirmation:** the editor asks for developers' confirmation before performing some action.
12. **Effect configuration:** the developer configures an effect to use it in damage types
13. **Damage type configuration:** the developer configures the type of damage.
14. **Tower type configuration:** the developer sets the damage type, attack range, attack cooldown, upgrade/sell system, active/inactive modes of tower entities.
15. **Enemy type configuration:** the developer sets enemy speed, HPs amount, texture and killing reward.

16. **Ability configuration:** configuring the ability (its type and parameters of this ability type)
17. **Grid and map layout setup**: configuring tile-based grid properties and background assets of the level map.
18. **Enemy path configuration:** defining and drawing the route trajectories and movement waypoints that enemies will follow. Setting reference to the HP counter, associated with this path
19. **Entity placement and spawning**: selecting game entities from a list and placing them onto the map.
20. **Move entity on the map**
21. **Entity duplication:** cloning an existing entity along with all its configured fields.
22. **Group operations**: selecting multiple entities to move, duplicate or delete them in group.
23. **Export game:** the developer chooses the directory to which he wants to export his game and the editor exports the game there by:
    a. Creating game files structure
    b. Exporting entities
    c. Generating level configuration
    d. Creating game executable
24. **Import game:** the developer imports previously exported game so he can edit it inside of the editor.
25. **Launch the game:** the user launches the game using the generated executable.
26. **Place a tower:** the player selects a tower type and places it in an available position on the map.
27. **Sell a tower:** the player sells a selected tower and receives part of its cost back.
28. **Upgrade a tower:** the player upgrades a selected tower using available resources.

29. **Use an ability:** the player activates an available special ability during the game.
30. **Start the next wave:** the player starts the next enemy wave.
31. **Change game speed:** the player increases or decreases the game speed
32. **Kill an enemy**

Detailed Descriptions

1. Create a new project

Actors: developer
Goal: create a new project
Precondition/Context: The editor is running
Main success scenario:

1. The developer clicks the “New project” button in the editor menu
2. Editor creates a new project and initialize new in-memory game descriptor (with one empty level descriptor and empty global entities list)
3. Editor opens the empty level associated with new project for editing

2. Load texture

Actors: developer
Goal: load base or custom texture for towers, enemies, strategic assets, GUI elements, map background or projectiles.
Trigger condition: developer opens textures/assets panel in the editor.
Precondition/Context: The editor is running, a level is opened.

Extensions:

- (step 3, see condition in step 3, **Handle error**)

Main success scenario:

1. Developer selects a texture category.
2. Developer imports a base or custom texture file and chooses the texture name. If file system error occurs, the selected file has an unsupported format or a texture with the same name already exists, **Handle error** with appropriate error message, and abort importing
3. Editor assigns the texture to the selected category.
4. Editor previews the texture.
5. Developer saves the changes.
6. Editor writes the texture reference into the in-memory project descriptor.

3. Load font

Actors: developer
Goal: load base or custom font for GUI.
Trigger condition: developer opens fonts panel in the editor.
Precondition/Context: The editor is running, a level is opened.
Extensions:

- (step 3, see condition in step 3, **Handle error**)

Main success scenario:

1. Developer imports a base or custom font file and chooses the font name. If file system error occurs, the selected file has an unsupported format or a font with the same name already exists, **Handle error** with appropriate error message, and abort importing
2. Developer saves the changes.

3. Editor writes the font reference into the in-memory project descriptor.

4. Remove a texture

Actors: developer
Goal: remove a texture.
Trigger condition: developer opens textures/assets panel in the editor and presses “remove” button by one of the textures name
Precondition/Context: The editor is running, a level is opened.
Main success scenario:

1. The editor removes all references to this texture from all game entities
2. The editor removes the texture reference from the in-memory project descriptor

5. Remove a font

Actors: developer
Goal: remove a texture.
Trigger condition: developer opens fonts panel in the editor and presses “remove” button by one of the fonts name
Precondition/Context: The editor is running, a level is opened.
Main success scenario:

1. The editor removes all references to this font from all game entities
2. The editor removes the font reference from the in-memory project descriptor

6. Create an entity in project-global entities list

Actors: developer

Goal: create an entity

Trigger condition: developer press a button “new entity” in the editor menu

Precondition/Context: editor is running, a level is opened.

Main success scenario:

1. Developer selects the entity type (e.g. tower type, enemy type, some type of effect) from the presented list.
2. The editor adds a new entity to the project-global list of entities.

7. Create a level entity

Actors: developer
Goal: create an entity
Trigger condition: developer press a button “+” corresponding to a level entity reference property of some of existing level entities (or game level) editing menu
Precondition/Context: editor is running, a level is opened.
Main success scenario:

1. Developer selects the entity type (e.g. EnemyWave or TowerSpot) from the presented list.
2. The editor adds a new entity to the level list of entities and places a reference on it inside of level entity reference property.

8. Remove an entity

Actors: developer
Goal: remove an entity.
Trigger condition: developer presses “remove” button by one of the entities in list (either project-global or level) of entities
Precondition/Context: The editor is running, a level is opened.
Main success scenario:

1. The editor removes all references to this entity from all game entities
2. The editor removes the entity from the in-memory level/project descriptor

9. Configure wave entity

Actors: developer

Goal: configure enemy waves by setting wave duration and next wave for wave, adding some enemy paths to list of paths and setting spawn rate, enemy types for each enemy path.

Precondition/Context: editor is running, a level is opened, enemy paths are available, enemy entity types are available.

Main success scenario:

1. Developer opens one of wave entities' editing menu
2. Developer adds an enemy path to list of paths involved in wave
3. Developer sets the enemy entity types for the added path
4. Developer sets the spawn rate for the added path
5. If developer wants to add one more enemy path to the wave, go to step 2
6. Developer sets wave duration and next wave reference (or none) for the wave
7. Developer saves the wave configuration

10. Handle error

Actors: developer

Goal: get the information about an error occurred during editing or configuration.

Precondition/Context: editor is running, the developer is performing an erroneous operation in the editor.

Main success scenario:

1. The editor creates a new window with:
    a. The error message displayed
    b. The “OK” button
2. The developer presses the “OK” button and the window closes

11. Get confirmation

Actors: developer
Goal: insure correctness of the operation that will be performed be the editor and possibly cancel it
Main success scenario:

1. The editor creates a new window with:
    a. The info about consequences of the operation that will be performed
    b. The “Proceed” button
    c. The “Cancel” button
2. If the developer presses “Proceed” button, the window closes and editor proceeds with the operation
3. Else, the window closes and editor doesn’t proceed with the operation

12. Effect configuration

Actors: developer

Goal: configure an effect used for creating damage types.

Precondition/Context: editor is running, a level or is opened.

Main success scenario:

1. Developer opens one of effect entities’ editing menu

2. Developer chooses effect type (damage, poisoning, slowness)
3. Developer configures parameters of the effect
4. Developer saves the effect configuration

13. Damage type configuration

Actors: developer

Goal: configure a type of damage used by game entities.

Precondition/Context: editor is running, a level or project is opened.

Main success scenario:

1. Developer opens one of damage type entities' editing menu
2. Developer adds references to effect entities associated with this damage type
3. Developer sets damage kind (projectile, instant reach, homing projectile)
4. Developer assigns texture for damage type
5. Developer saves the damage type configuration

14. Tower type configuration

Actors: developer
Goal: configure a type of tower that can be placed by player
Precondition/Context: the editor is running, textures for the tower and its projectiles have been loaded.
Main success scenario:

1. Developer opens one of tower type entities' editing menu

2. The developer assigns texture for the tower body and selects damage type;
3. The developer configures attack range and cooldown;
4. The developer add references to tower type entities to which this tower can be promoted and assigns price for such promotions
5. The developer previews the tower in context (range circle);
6. Developer saves the tower type configuration

15. Enemy type configuration

Actors: developer
Goal: configure an enemy type that can be spawned during wave
Precondition/Context: the editor is running, textures for the enemy have been loaded.
Main success scenario:

1. Developer opens one of enemy type entities' editing menu
2. The developer assigns texture for the enemy body
3. The developer configures enemy speed
4. The developer configures enemy HPs amount
5. The developer sets reward size for killing this enemy
6. The developer optionally adds multipliers for different effects that can affect enemies of this type (so the developer configures strengths and weaknesses)
7. Developer saves the enemy type configuration

16. Ability configuration

Actors: developer
Goal: configure an ability that can be used by player
Precondition/Context: the editor is running
Main success scenario:

1. Developer opens one of ability entities' editing menu
2. The developer choose the type of the ability (healing of player, damage ability)
3. The developer configures parameters of the selected ability type
4. Developer saves the ability configuration

17. Grid and map layout setup

Actors: developer
Goal: configuring tile-based grid properties and background assets of the level map.
Trigger condition: the developer opens the map/grid settings panel.
Precondition/Context: the editor is running, a level is opened, background textures have been loaded (or defaults are available).
Main success scenario:

1. The developer opens the grid and map layout settings panel;
2. The developer configures grid properties (cell size, map width/height in cells);
3. The developer assigns background assets to the map;
4. The developer previews the map layout in the editor viewport;
5. The developer saves; the editor writes grid, background data into the in-memory level descriptor.

18. Enemy path configuration

Actors: developer
Goal: define and draw route trajectories with waypoints that enemies follow path start to path end.

Precondition/Context: the editor is running, a level is opened, map layout is set up

Main success scenario:

1. Developer opens one of enemy path entities' editing menu, presses "draw a path" button and enters path editing mode
2. The developer places sequential waypoints on the grid to define the movement trajectory
3. When the arrangement is complete, the developer clicks the "complete editing" button and exits the special editor mode.
4. The engine changes enemy path entity state

19. Entity placement and spawning

Actors: developer

Goal: select placeable game entities (tower spots, ui elements) from a list of entities and place them onto the map.

Trigger condition: the developer opens the entity list in the level editor.

Precondition/Context: the editor is running, there is at least one placeable entity, the map layout is set up.

Main success scenario:

1. The developer selects an entity from the entity list;
2. The developer moves the cursor over the map and places the entity;
3. Game engine change this entities' state

20. Move entity on the map

Actors: developer

Goal: Move entity on the map

Precondition/Context: the editor is running, there is at least one entity on the map.
Main success scenario:

1. The developer selects an entity on the map;
2. The developer moves the cursor over the map and places the entity;
3. Game engine change this entities' state

21. Entity duplication

Actors: developer
Goal: clone an existing entity along with all its configured fields.
Precondition/Context: the editor is running, a level is opened, there is at least one entity placed on the map
Main success scenario:

1. The developer selects an entity on the map or the entity list;
2. The developer invokes "duplicate";
3. Editor creates a copy of the entity with all fields value copied;
4. Editor places the clone at an offset position;

22. Group operations

Actors: developer
Goal: select multiple entities to move, duplicate or delete them.
Precondition/Context: the editor is running, a level is opened, multiple entities exist on the map.
Main success scenario:

1. The developer selects multiple entities;
2. The developer invokes a group action: move, duplicate or delete;
3. The editor changes state of the entities

23. Export game

Actors: developer
Goal: export game to some directory, so it can be distributed to players, launched or imported back to the same game editor.
Trigger condition: the developer presses the “export” button in the editor menu.
Precondition/Context: the editor is running, a level is opened
Extensions:

- (step 3, see condition in step 3, **Get confirmation**)
- (steps 8-10, see condition in step 8-10, **Handle error**)

Main success scenario:

1. The editor creates a window, where developer can choose the export directory
2. The developer choose some export directory
3. If the export directory isn’t empty, call “Get confirmation” use case to ask for confirmation to clear the export directory before export
4. Else, got to step 7
5. If the developer cancel the operation, go to the step 2 and wait for another export directory chosen
6. Else, clear the export directory and go to step 7
7. The editor closes the window created on step 1
8. The editor creates appropriate directories inside the export directory. If a file system error occurs, rollback all changes in file system made by game editor during export and call **Handle error** UC with appropriate message
9. The editor exports all game textures to the appropriate subdirectory of the export directory. If a file system error occurs, rollback all changes in file system made by game editor during export and call **Handle error** UC with appropriate message

10. The editor serializes all game entities to file/files inside of the appropriate subdirectory of the export directory. These files have some format recognizable by both the editor and the game engine executable. If a file system error occurs, rollback all changes in file system made by game editor during export and call **Handle error** UC with appropriate message
11. The editor generates a level description as a configuration file of the format recognizable by both the editor and the game engine executable. If a file system error occurs, rollback all changes in file system made by game editor during export and call **Handle error** UC with appropriate message

24. Import game

Actors: developer
Goal: edit previously exported game in the editor
Trigger condition: the developer presses the “import” button in the editor menu.
Precondition/Context: the editor is running.
Extensions:

- (step 4, see condition in step 4, **Handle error** with appropriate message)

Main success scenario:

1. The editor creates a window, there developer can choose the directory from which to import the game
2. The developer choose some directory
3. The editor reads game files in the directory
4. If a file system error occurs, game files have invalid format or some game file is corrupted (e.g. have null references), call **Handle error** UC with appropriate message, after that

the game editor waits for the user to choose another directory or exit.
5. Else the editor opens the project for editing

25. Launch the game

Actors: player
Goal: launch the game
Trigger condition: the player executes game engine executable
Precondition/Context: the player has game engine installed, the player has the game files exported by the game editor
Main success scenario:

1. The game engine creates a window, there player can choose the directory with game files exported be the game editor
2. Player chooses some directory
3. The engine reads game files from the chosen directory.
4. If a file system error occurs, game files have invalid format or some game file is corrupted (e.g. have null references), the engine displays appropriate error message and waits for the user to choose another directory or exit.
5. Else the game engine starts the game

26. Place a tower

Actors: player
Goal: place a tower of specific type in an available position on the map to defend against enemies.
Trigger condition: player opens the tower selection UI
Precondition/Context: game is running; the map grid is visible; there is at least one empty tower spot
Main success scenario:

1. Player clicks on an empty tower spot

2. A tower actions menu appears near the tower, the upgrade button displays as active, the "sell" button is displayed as inactive
3. Player clicks on “upgrade tower” button in the tower actions menu
4. The tower actions menu closes and a tower upgrade menu with a list of available upgrade options opens. If player have sufficient amount of money for the specific upgrade, the button “Upgrade”, corresponding to it displayed as active, else it’s displayed as inactive
5. Player clicks the specific active "Upgrade" button corresponding to the desired upgrade option.
6. The upgrade tower menu closes and new tower appears in the spot

27. Sell a tower

Actors: player
Goal: remove a placed tower from the map and receive a portion of its cost back
Precondition/Context: game is running, at least one tower is placed on the map
Main success scenario:

1. Player clicks on a placed tower
2. A tower actions menu appears near the tower
3. Player clicks the "sell" button in the tower actions menu
4. The tower actions menu closes and player gets money

28. Upgrade a tower

Actors: player
Goal: improve a selected tower's stats
Precondition/Context: game is running
Main success scenario:

1. Player clicks on a placed tower
2. A tower actions menu appears near the tower
3. If this tower has any upgrade options, the upgrade tower button is displayed as active.
4. Else, the upgrade tower button is displayed as inactive and player cannot proceed with upgrading (**end of UC**)
5. Player clicks on “upgrade tower” button in the tower actions menu
6. The tower actions menu closes and a tower upgrade menu with a list of available upgrade options opens. If player have sufficient amount of money for the specific upgrade, the button “Upgrade”, corresponding to it displayed as active, else it’s displayed as inactive
7. Player clicks the specific active "Upgrade" button corresponding to the desired upgrade option.
8. The upgrade tower menu closes and tower changes its' type

29. Use an ability

Actors: player
Goal: damage the enemies
Precondition/Context: game is running, the ability is not on cooldown
Main success scenario:

1. Player clicks on the ability icon in the UI
2. If the ability requires targeting, player clicks on the target location on the map to confirm activation.
3. Ability activates

30. Start the next wave

Actors: player
Goal: make progress in completing a game level
Trigger condition: player clicks the “Call Next Wave” button
Precondition/Context: game is running; the previous wave ended
Main success scenario:

1. Player clicks the "Start Next Wave" button in the UI or presses the designated hotkey for starting the next wave (e.g., Spacebar or 'N').
2. The next wave starts

31. Change game speed

Actors: player
Goal: increase or decrease the simulation speed of the game.
Precondition/Context: game is running
Main success scenario:

1. Player clicks the increase or decrease button in the UI or presses the designated hotkeys to cycle through or set the game speed multiplier (e.g., '+', '-', or '1', '2', '3')
2. The game speed multiplier changes to chosen value

32. Kill an enemy

Actors: player
Goal: earn money and protect strategic asset (players’ HPs)
Precondition/Context: game is running, there is at least one enemy on the map

Main success scenario:

1. Tower or player (with ability) deliver damage to the enemy
2. Enemies’ HPs lower to zero
3. Enemy disappears from the map
4. The player earns some sum of money (depending on game rules)